

מפתח ניקוד לתרגיל בית 5

סימון	הערה	ניקוד
W	לא הוגש	100
YYY	פייתון F5 - לא עובד, אבל לאחר תיקון מינימלי התוכנית פועלת. ניקוד על סעיף זה לא יוחזר ואין לערער עליו.	15
שאלות		
שאלה 1		15
1A	תשובה לא נכונה לסעיף א'. התשובה הנכונה (בקצרה) היא: אתחול השדה self.rows בצורה הנ"ל הינו בעייתי משום שהוא יוצר מטריצה בה השורות מצביעות לאותו מקום בזכרון. כך למשל, אם נרצה לשנות איבר ספציפי בשורה r ובעמודה c, נקבל תוצאה לא רצויה בה האיבר בעמודה c בכל אחת מהשורות השתנה גם כן.	5
1A*	הסבר חלקי/ לא הוספו דוגמאות הרצה לסעיף א'.	2
שאלה 2		25
שאלה 3		15
שאלה 4		25
4D	תשובה לא נכונה לסעיף ד'. התשובה הנכונה היא תשובה מספר (2).	5
4D*	הסבר חלקי/ לא מספק לסעיף ד'. התשובה הנכונה היא: על מנת להחזיר את ההפרש המקסימלי בין גובהם של שני בנים (של אותו אב) בעץ, נצטרך לפי שיטתו של אמיר לחשב לכל צומת node את עומקו (עם המתודה שהוצעה על ידו) אשר פועלת בסיבוכיות של $O(k)$ (כאשר k מספר הצמתים בתת העץ ששורשו הוא הצומת node). לכן, אנחנו עלולים לקבל סיבוכיות גבוהה מ- $O(n)$.	2
שאלה 5		20
5C	תשובה לא נכונה לסעיף ג'. האורכים הנכונים הם: 56, 65, 82, 53, 146, 86.	5
5C*	הסבר חלקי/ לא מספק לסעיף ג'.	2
כללי		
P1	שימוש בשמות משתנים לא משמעותיים המקשים על הבנת הפתרון	2
P2	הדפסות מיותרות	5

5	צירוף פתרון בכתב יד שאינו קריא	P3
(-:	תשובה לשאלה i טובה ומקורית	Vi
ניקוד	הערה	סימון
קבצים		
5	הגשה לא תקנית (zip וכו')	Z1
5	שם אחד הקבצים לא בפורמט הנכון	Z2
5	הגשת קובץ docx במקום קובץ pdf	Z3
טסטר		
(*)	תשובה לא נכונה בבדיקה k של שאלה i סעיף j	Tij_k
(*)	הפונקציה של שאלה i סעיף j לא עוברת אינטרפטציה ע"י IDLE	Tij_X
(*)	הפונקציה של שאלה i סעיף j חרגה מזמן הריצה המוקצה	Tij_t
5	שאלה 2 - מחיקה של איבר בעל ערך דיפולטי במטריצה דלילה ללא בדיקה האם האיבר נמצא במילון	T_del
5	שאלה 5 - הרצה אוטומטית (ולא דרך פונקציה ייעודית) של החיפוש הבינארי הנדרש בשאלה	T_gen

(*) ניתן לראות את הניקוד ע"י הרצת הטסטר.

הערות לתרגיל

שאלה 2: סטודנטים רבים הבחינו נכונה כי כאשר מבצעים השמה של ערך דיפולטי במטריצה דלילה (כלומר לבצע השמה של הערך 0 בשורה ה-i ית ועמודה ה-j ית של המטריצה) אנו לא מעוניינים להוסיף את הערך הנ"ל למילון. עם זאת, יש להבחין בין שני מקרים אפשריים:

- אם המטריצה הוגדרה עם ערך שונה בנקודה זו יש למחוק את הערך מהמילון.
- אחרת, אין לבצע שינוי במבנה הנתונים כלל.

במספר מקרים נעשה ניסיון למחוק את הנקודה הרלוונטית מהמילון ללא בדיקה מוקדמת של השמה בנקודה זו, מה שגרר שגיאת זמן ריצה בשל גישה לאיבר שלא מוגדר במילון.

שאלה 3ב': בעת הרכבת פונקציות נרצה לוודא שתחום ההגדרה של הפונקציה החדשה מוגדר היטב. בחישוב עבור פונקצית ההרכבה: $h(x) = (f \circ g)(x) = f(g(x))$ נרצה ראשית לבדוק כי g מוגדרת בנקודה x ולאחר מכן לבדוק כי f מוגדרת בנקודה g(x). כלומר, המימוש הנכון

יתבצע כך: `dom_comp = (other.domain(x) and self.domain(other.func(x)))`. שימו לב לסדר הבדיקה ולקלטים הנשלחים לפונקציה `domain`.

- שאלה 4ד': כמעט הרוב המוחלט של הסטודנטים זיהה בהצלחה את המשפט הנכון מבין שלוש האפשרויות. עם זאת, ישנם סטודנטים שביצעו ניתוח שגוי של סיבוכיות המתודה המוצעת בסעיף. סיבוכיות הזמן של המתודה היא $O(n^2)$ (כאשר n מספר הצמתים בעץ).

הערות כלליות

- בעת כתיבת פתרון לשאלה, במיוחד בשאלות רבות-ברירה, הקפידו לציין קודם מה היא תשובתכם (ולא רק לצרף הסבר).
- צירוף תיעוד לקוד מתבצע באנגלית - כמו שפיתוח הקוד מתבצע באנגלית בלבד, כך גם ההערות אליו. נוסף על כך, פייתון אולי מצליח "לעכל" הערות בעברית על מכונה מסויימת, אבל אין הבטחה כי הרצה של הקוד על מכונה אחרת תצליח אף היא.
- שמות משתנים - משתנים בעלי אות בודדה (כגון 'i') לרוב משמשים כאינדקסים בלולאה או פרמטרים מספריים. אחרת, נעדיף לקרוא למשתנה לפי מה שהוא מייצג. לדוגמה, לרשימה נקרא 'lst' (ולא 'l') או למילון נקרא 'dict' (ולא 'd'). כזכור, שם המשתנה אמור להעיד על מטרתו (חשוב כמובן לא להגזים...).
- שימוש בקבועים - קבועים הופכים את הקוד לנקי וברור יותר לקריאה. דוגמה לשימוש בקבוע: אם נרצה לבצע סימולציה של מספר הרצות לפונקציה מסוימת, נשמור משתנה `NUM_SIMULATIONS` (מקובל לתת לקבוע שם באותיות גדולות) ונאתחל אותו כרצוננו. לשימוש בקבוע יש מס' יתרונות: שמו מעיד בקלות על מטרתו והוא ניתן לשינוי בקלות (משום שהשינוי יצטרך להתבצע רק במקום אחד בקוד).

משוב סגנון - שאלה 4 סעיף ב' - store_heights()

שמות הקלאסטרים מופיעים בטבלה המצורפת מטה. במשוב האישי המצורף ב-moodle ניתן למצוא גם את שם הקלאסטר שאליו תשובתכם סווגה, אך מומלץ לעבור על המסמך כולו.

בשאלה הנתונה נתבקשתם להוסיף למחלקה Tree_node שדה שישמור את גובה הצומת בעץ, ולאחר מכן לבנות מתודה רקורסיבית למחלקה Binary_tree_node שתשמור לכל צומת בעץ את גובהו (על מנת שתוכלו אחר-כך לממש ביעילות את הפונקציה height_diff). מרבית התשובות שהוגשו החזירו את התשובה הנכונה, אך זאת הזדמנות טובה לבחון את הדרכים השונות להתמודד עם האתגר של ניתוח מקרי הקצה בפונקציות רקורסיביות.

קבוצה	משוב סגנון
CA	השתמשותם בתנאי עצירה בודד. פתרון קלאסי ויפה. דוגמה למימוש כנ"ל: <pre>def store_heights(self): def height_rec(node): if node == None: return -1 lh = height_rec(node.left) lr = height_rec(node.right) node.height = max(lh,lr)+1 return node.height height_rec(self.root)</pre> הפונקציה הפנימית מבצעת שתי פעולות: עדכון גובה הצומת הנוכחי עליו אנחנו עומדים (מלבד צמתי None) ובנוסף שליחת הגובה אותו זה עתה חישבנו במעלה הרקורסיה, על מנת להשתמש בערך זה כדי לקבוע את גובהם של צמתים גבוהים יותר בעץ. מבחינת תנאי הקצה, קיים מקרה קצה יחיד עבור המצב בו node == None. קל לראות שנגיע למקרה זה רק בצמתים שאינם מלאים (כלומר שהם עלים או שאחד הבנים שלהם לא קיים) ובמקרה זה נחזיר עבור צמתי None אלה את הערך 1-1. נשים לב שהחזרת ערך כזה עקבית עם הגדרת מושג הגובה שלמדנו: לדוגמה עבור צומת עלה (שלא קיימים לו בנים) נקבל שערך הגובה שלו (לפי הפונקציה שלנו) הוא: $0 = 1 + \max(-1, -1)$, כפי שרצינו. לבסוף, ניתן לומר שהפונקציה כמעט "כותבת את עצמה", שכן בהינתן גובהם של שני בניו של צומת מסוים, יקבל הצומת את הגובה המקסימלי מביניהם + 1.
CB	ביצעתם מימוש דומה לקבוצה הקודמת - כלומר השתמשם גם במקרה קצה אחד. עם זאת, בוצעה החזרה של 0 ולא של 1-1. ראינו כי על מנת לשמור על קונסיסטנטיות עם הגדרת גובהו של צומת בעץ עלינו עלינו להחזיר 1-1 עבור צמתי None.

קבוצה	משוב סגנון
CC	השתמשותם ביותר מתנאי עצירה אחד. דוגמה למימוש כנ"ל: <pre>def store_heights(self): def store_heights_rec(node): if node.left==None and node.right==None: node.height=0 return node.height elif node.left!=None and node.right!=None: node.height=1 + max(store_heights_rec(node.left), store_heights_rec(node.right)) return node.height elif node.right==None: node.height=1+ store_heights_rec(node.left) return node.height else: node.height= 1+store_heights_rec(node.right) return node.height store_heights_rec(self.root)</pre>
	ראשית נציין שמימוש זה הוא תקין (למעט העובדה שלא מטפל במקרה הקצה בוא נשלח לפונקציה "עץ ריק" - כלומר <code>self == None</code> כבר מראש). קל לראות את ההבדל בין שני המימושים לעיל: מספר תנאי העצירה. הרעיון העומד בבסיסו של מימוש זה שתנאי העצירה של הפונקציה מטפלים בצמתים "שלב אחד קודם" לעומת המימוש הקודם (ולכן מספר תנאי העצירה גדלו). ניתן לראות שישנה הפרדה מהו סוג הצומת הנוכחי עליו אנחנו עומדים: צומת עלה (ששני בניו הם None), צומת עם בן אחד (בן אחד שווה לNone והשני לא) או צומת מלא (ששני בניו שונים מNone). עם זאת, בניתוח המימוש הקודם ראינו כיצד ניתן להסתפק בתנאי עצירה אחד. ודאו כי אתם מבינים כיצד כל אחד מתנאי העצירה במימוש הנ"ל היה מטופל במימוש הקודם.
CD	מימושים דומים לקבוצה CC - שימוש ביותר מתנאי עצירה אחד.
CE	לא מימשותם את הפונקציה הנ"ל.
NOC	ישנם סטודנטים שלא סווגו לאף קבוצה. אין זה מעיד על נכונות/ אי נכונות הפתרון. אתם יכולים לעיין בשאר הקבוצות ולבדוק אם אתם עונים לדרישות של אחת הקבוצות.